

如何在Java中实现手写数字识别神经网络

前言

数据源来源[MNIST](#)手写数据库，其中有60000个数据作为训练集，10000个数据作为验证集，再使用Adam优化器、L1+L2正则化的训练后，准确率能到达95%左右（不是很高，哈哈，很多优化的地方还没写好）。

如果你和我一样刚接触神经网络，推荐你看这些视频，然后你就会理解我下面讲的反向传播的内容了。

这些视频在我学习期间帮助了我很多，他们并没有晦涩难懂的部分！（按照阅读顺序）

1. [\[3B1B\] 深度学习之神经网络的结构 Part 1 ver 2.0](#)
2. [\[3B1B\] 深度学习之梯度下降法 Part 2 ver 0.9 beta](#)
3. [\[3B1B\] 深度学习之反向传播算法 上/下 Part 3 ver 0.9 beta](#)
4. [梯度下降算法](#)
5. [反向传播算法](#)
6. [反向传播可视化](#)
7. [Adam 优化器](#)
8. [L1和L2正则化](#)

需要提前说明的是，这篇文档中，很多专业名词性的解释可能不到位，也可能漏洞百出。因此希望大家谅解。如果可以的话，请帮我更正！此外，这篇文档会非常非常详细地讲解如何从0构建手写数字识别的神经网络，篇幅较长。

最后，我诚恳的希望诸位能指出我逻辑上的错误，然后在Issue发布您的更正，感谢。

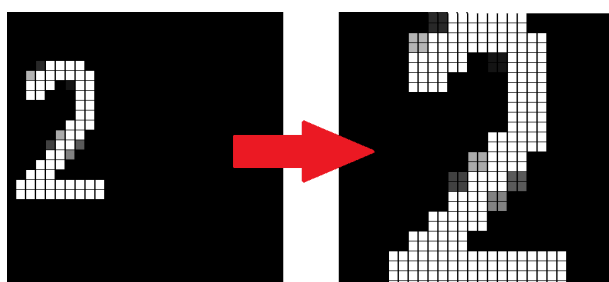
图像处理

首先，不论是训练集还是验证集，[MNIST](#)提供的手写图像都是 $28 \times 28 = 784$ 像素的。也就意味着我们需要利用这784个像素作为输入层。

但是在这些像素传入输入层之前，我们先要对图像进行非常简单的预处理。

为什么要进行预处理？因为假设我写了一个非常偏左的数字2，我们将其直接预测，效果会非常不好，因为神经网络对于在边缘的数字识别能力不强。因此，我们对训练集进行预处理、对在画板上的数字也进行预处理，让他们相互集中在中间，这样会**非常明显地**提高识别准确度！

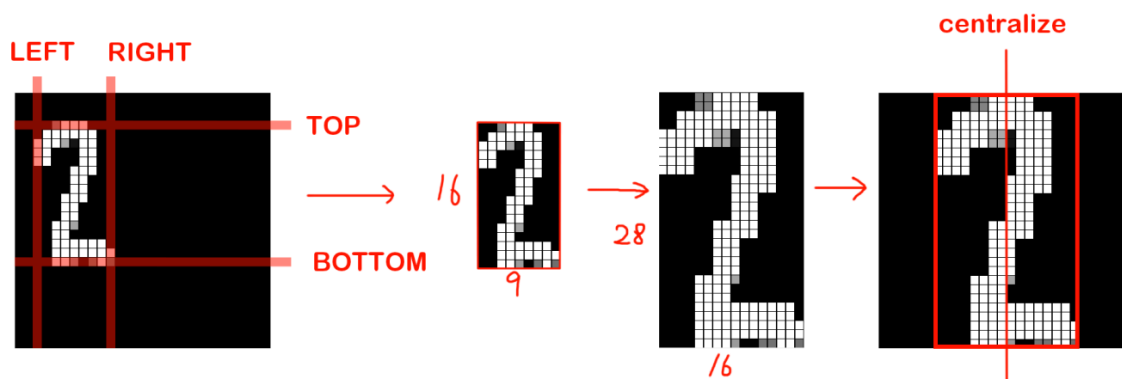
例如，我书写了数字2，但是由于偏左，我们不仅需要把它挪动至中间，并且需要放大到28*28。这样我们才能用一个比较统一的标准来进行训练以及预测。



对于一个输入的图像，我们首先要获取它的最小图像，也就是从 left 到 right，从 top 到 bottom。
(如下图)

然后我们抽取出这个图像，其宽度为 $\text{Math.abs}(\text{bottom} - \text{top}) + 1$ 长度为 $\text{Math.abs}(\text{left} - \text{right}) + 1$ ，存入数组A中。

```
int height = Math.abs(bottom - top) + 1;
int weight = Math.abs(left - right) + 1;
double[][] A = new double[height][weight]; // 按照我们的例子，现在数组A的大小应当为
A[16][9]
```



接下来，我们尽可能把A图像放大。A数组的较长边被拉伸成28(和 28×28 匹配)，在这个例子中，是宽度16被拉伸成28，因此，

$$k = \frac{28}{width_{old}}$$
$$width_{new} = width_{old} \times k = 16 \times \frac{28}{16} = 28$$
$$length_{new} = length_{origin} \times \frac{28}{weight_{origin}} = 9 \times \frac{28}{16} = 15.57 = (int)16$$

在这里我们把拉伸后的图像视为

```
double[][] B
```

然后，我们让B居中即可完成位置的初始化。

接下来，我们要对数据进行归一化，也就是把所有的值归一化到 $[0.0, 1.0]$ 的范围。这样能增强表达。

在整个 28×28 的图片中，假设像素激活值 $([0, 1])$ ，白色像素是1，黑色像素是0) 最小值是min，最大值是max，同时保证获取到的最小值min≠画布的0.0(黑色)，而是 > 0 的最小值。然后对于这样的像素值 i_{old} 我们归一化如下

$$i_{new} = \frac{i_{old} - min}{max - min}$$

例如，现在我们的图像中最小值是0.1, 最大值是0.7。那么对于任意一个像素 i_{old} 其更新后应该为

$$i_{new} = \frac{i_{old} - 0.1}{0.7 - 0.1}$$

代码我们可以这样写

```

for (int x = 0; x < currentImg.length - 1; x++) {
    for (int y = 0; y < currentImg[0].length; y++) {
        currentImg[x][y] = (currentImg[x][y] - min) / (max - min);
    }
}

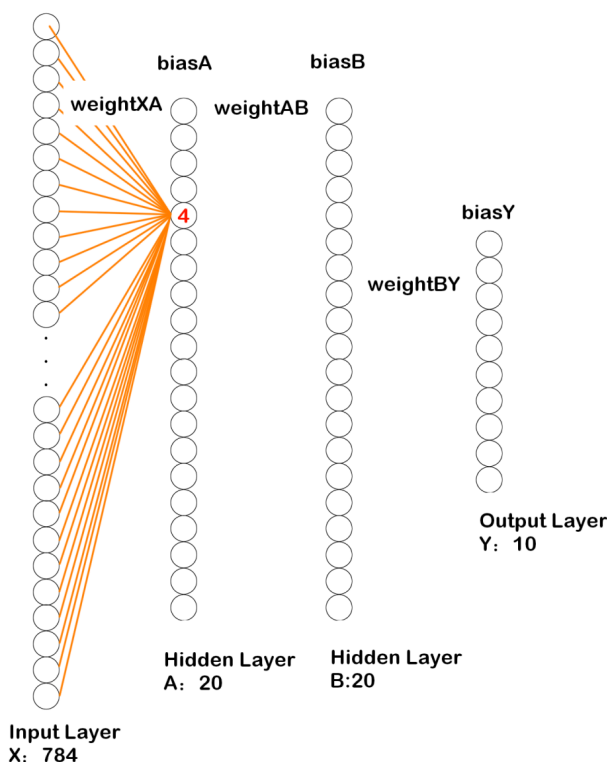
```

然后就完成了一个图像的初始化。

向前传播

接下来是这个神经网络如何向前传播。

神经网络总体结构如下图所示



首先，让我们看看这个神经网络的构成：

输入层 **X**: 784个神经元，约定下标为 l 。

隐藏层1 **A**: 20个神经元，约定下标为 r 。

隐藏层2 **B**: 20个神经元，约定下标为 i 。

输出层 **Y**: 10个神经元，约定下标为 j 。（也就是数字1~9共10个输出）

这四个层命名如下

```

double[] x = new double[784]; // 输入层
double[] A = new double[20];  // 隐藏层
double[] B = new double[20];  // 隐藏层
double[] Y = new double[10];  // 输出层

```

同时还有权重(weight)设置

1. X层对A层的权重 $weight_{XA}[784][20]$ ，数学符号 $W_{l,r}^{(1)}$

2. A层对B层的权重 `weightAB[20][20]` , 数学符号 $W_{r,i}^{(2)}$

3. B层对Y层的权重 `weightBY[20][10]` , 数学符号 $W_{i,j}^{(3)}$

偏置(bias)设置

1. 偏置A层 `biasA` , 数学符号 $Bias_r^{(1)}$

2. 偏置B层 `biasB` , 数学符号 $Bias_i^{(2)}$

3. 偏置Y层 `biasY` , 数学符号 $Bias_j^{(3)}$

```
static double[][] weightXA = new double[784][20]; // 下标 weightXA[l][r]
static double[][] weightAB = new double[20][20]; // 下标 weightAB[r][i]
static double[][] weightBY = new double[20][10]; // 下标 weightBY[i][j]
static double[] biasA = new double[20]; // 下标 biasA[r]
static double[] biasB = new double[20]; // 下标 biasB[i]
static double[] biasY = new double[10]; // 下标 biasY[j]
```

值得注意的是, 权重和偏置的值在最开始是随机的 (具体来说是**均匀分布的He初始化**, 这个最后说)。

对于**权重** `weight` 的数学符号, 我们这样约定

$$W_{l,r}^{(1)}$$

其中(1)代表第一层权重, 也就是 `weightXA` , l, r 代表左边点 l 连接到右边点 r 的权重。

对于**偏置** `bias`的数学符号, 我们这样约定

$$Bias_i^{(1)}$$

这代表第一层偏置 `biasA` 的第*i*个值。

例子:

符号	含义	符号	含义
$W_{740,3}^{(1)}$	<code>weightXA[740][3]</code> X层点740到A层点3的权重	$Bias_{13}^{(1)}$	<code>biasA[13]</code> A层点偏置
$W_{16,2}^{(2)}$	<code>weightAB[16][2]</code> A层点16到B层点2的权重	$Bias_7^{(2)}$	<code>biasB[7]</code> B层点偏置
$W_{20,8}^{(3)}$	<code>weightBY[20][8]</code> B层点20到Y层点8的权重	$Bias_4^{(3)}$	<code>biasY[4]</code> C层点偏置

那么, 图中 `A[4]` 怎么求呢?

$$a_4 = (x_1 \quad x_2 \quad \cdots \quad x_{784}) \begin{pmatrix} W_{1,4}^{(1)} \\ W_{2,4}^{(1)} \\ \vdots \\ \cdot \\ W_{783,4}^{(1)} \\ W_{784,4}^{(1)} \end{pmatrix} + bias_4^{(1)}$$

```

for(int left=0;left<784;left++)
{
    A[4] += x[left] * weightXA[left][4]; // weightXA[left][right] 代表左边left
    连接到右边right的那条权重
}
A[4] += biasA[4]; // 添加偏置

```

在运算完后，我们需要让 a_4 的范围压缩到 $[0, 1]$ 之内，因此我们会用到sigmoid函数：

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

因此

```

A[4] = sigmoid(A[4]) // 压入sigmoid

```

这样一个神经元就计算好了。

总而言之，向前传播就是不断用矩阵相乘，然后加上偏置后压入sigmoid函数。

因此**向前传播**的代码就是

```

double[] x = new double[784];
double[] A = new double[20];
double[] B = new double[20];
double[] Y = new double[10];
// 计算隐藏层A
for (int i = 0; i < 20; i++) {
    for (int left = 0; left < 784; left++) {
        A[i] += x[left] * Processing.weightXA[left][i]; // Processing. 是本类中的变
        量。
    }
    // 添加偏置后，压入sigmoid
    A[i] += Processing.biasA[i];
    A[i] = sigmoid(A[i]);
}
// 计算隐藏层B
for (int i = 0; i < 20; i++) {
    for (int left = 0; left < 20; left++) {
        B[i] += A[left] * Processing.weightAB[left][i];
    }
    B[i] += Processing.biasB[i];
    B[i] = sigmoid(B[i]);
}
// 计算Y
for (int i = 0; i < 10; i++) {
    for (int left = 0; left < 20; left++) {
        Y[i] += B[left] * Processing.weightBY[left][i];
    }
    Y[i] += Processing.biasY[i];
    Y[i] = sigmoid(Y[i]);
}

```

传播完成后，假设我们输入了数字3，那么其输出层的结果如下所示。

```

[0] 55.91%
[1] 61.80%
[2] 70.88%
[3] 59.52% VERY BAD PREDICT!!!
[4] 47.61% :(
[5] 63.01%
[6] 22.50%
[7] 54.50%
[8] 52.99%
[9] 69.10%

```

显然，我们输入3，但是它预测的是2，而且每个数字概率好像也差不多。显然，这个输出非常差，因为我们还没有对神经网络进行训练！

那么，接下来的反向传播就会讲到如何利用梯度下降的方法对这个网络进行训练。

反向传播

先前，我们讲到向前传播大概是这样的：

$$X \rightarrow \alpha \rightarrow A \rightarrow \beta \rightarrow B \rightarrow \gamma \rightarrow Y$$

其中 α 、 β 、 γ 分别代表压入sigmoid前的值。也就是：

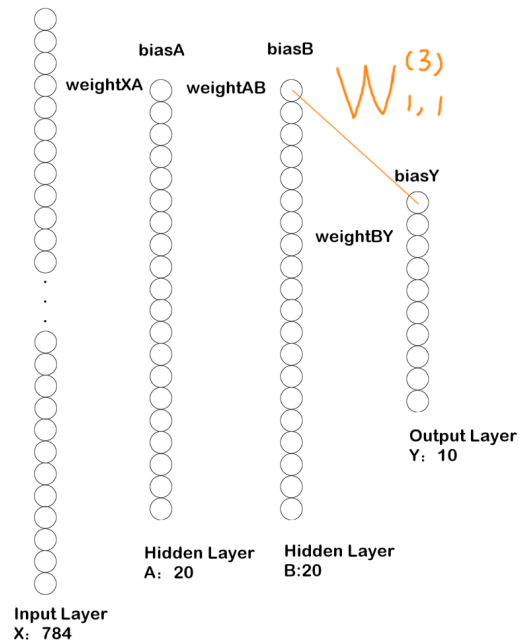
$$\alpha_4 = (x_1 \quad x_2 \quad \cdots \quad x_{784}) \begin{pmatrix} W_{1,4}^{(1)} \\ W_{2,4}^{(1)} \\ \vdots \\ \vdots \\ W_{783,4}^{(1)} \\ W_{784,4}^{(1)} \end{pmatrix} + bias_4^{(1)}$$

$$a_4 = \text{sigmoid}(\alpha_4)$$

我们这样定义是为了能够后续求链式法则更方便。

更新权重和偏置

更新 $W_{i,j}^{(3)}$



根据梯度下降公式,

$$W_{i,j}^{(3)'} = W_{i,j}^{(3)} - \epsilon \cdot \nabla G$$

$$\nabla G = \frac{\partial Loss}{\partial W_{i,j}^{(3)}}$$

$$Loss = \frac{1}{2} \sum_{j=1}^{10} (y_j - y_{gt(j)})^2$$

其中, ϵ 是学习率 (或者叫步长), 是一个常数。Loss是损失函数, 其中的 y_{gt} 是实际值。

例如, 我们写了一个6, 那么输出层 $y_{gt(6)} = Y[6] = 1.0$, 而其他则为 0.0。

y_j	$y_{gt(j)}$
[0] 63.85%	0.00
[1] 46.90%	0.00
[2] 76.71%	0.00
[3] 65.24%	0.00
[4] 70.42%	0.00
[5] 19.28%	0.00
[6] 40.07%	1.00
[7] 55.20%	0.00
[8] 69.67%	0.00
[9] 55.19%	0.00

因此,

$$W_{i,j}^{(3)'} = W_{i,j}^{(3)} - \epsilon \cdot \frac{\frac{1}{2} \sum_{j=1}^{10} (y_j - y_{gt(j)})^2}{\partial W_{i,j}^{(3)}}$$

首先我们先求梯度

$$\nabla G = \frac{\partial Loss}{\partial W_{i,j}^{(3)}} = \frac{\frac{1}{2} \sum_{j=1}^{10} (y_j - y_{gt(j)})^2}{\partial W_{i,j}^{(3)}}$$

但是这个式子不能直接求偏导，因此我们可以考虑链式法则。

在这里，我们还需要列出一些关系，这样才能求损失函数对 $W_{1,1}^{(3)}$ 的偏导。

$$y_j = \text{sigmoid}(\gamma_j)$$

$$\gamma_j = (b_1 \quad b_2 \quad \cdots \quad b_{20}) \begin{pmatrix} W_{1,j}^{(3)} \\ W_{2,j}^{(3)} \\ \vdots \\ W_{20,j}^{(3)} \end{pmatrix} + \text{Bias}_j^{(3)}$$

所以，

$$\begin{aligned} \nabla G &= \frac{\partial \text{Loss}}{\partial W_{i,j}^{(3)}} \\ &= \frac{\partial \text{Loss}}{\partial y_j} \frac{\partial y_j}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial W_{i,j}^{(3)}} \\ &= \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot \text{sigmoid}(\gamma_j)(1 - \text{sigmoid}(\gamma_j)) \cdot b_i \\ &= \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_i(1 - y_i) \cdot b_i \end{aligned}$$

注：对于 $\text{sigmoid}(x)$ ，其导数是 $\text{sigmoid}(x) \times (1 - \text{sigmoid}(x))$

因此，更新后的权重 W 应为：

$$W_{i,j}^{(3)'} = W_{i,j}^{(3)} - \epsilon \cdot \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_i(1 - y_i) \cdot b_i$$

更新 $\text{Bias}_j^{(3)}$

其实对于**biasY**而言，其更新是和上述差不多的。

$$y_j = \text{sigmoid}(\gamma_j)$$

$$\gamma_j = (b_1 \quad b_2 \quad \cdots \quad b_{20}) \begin{pmatrix} W_{1,j}^{(3)} \\ W_{2,j}^{(3)} \\ \vdots \\ W_{20,j}^{(3)} \end{pmatrix} + \text{Bias}_j^{(3)}$$

这里偏导最后项是1。

$$\begin{aligned}
 \nabla G &= \frac{\partial Loss}{\partial Bias_j^{(3)}} \\
 &= \frac{\partial Loss}{\partial y_j} \frac{\partial y_j}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial \textcolor{red}{Bias}_j^{(3)}} \\
 &= \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot \text{sigmoid}(\gamma_j)(1 - \text{sigmoid}(\gamma_j)) \cdot \textcolor{red}{1} \\
 &= \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j)
 \end{aligned}$$

因此,

$$Bias_j^{(3)'} = Bias_j^{(3)} - \epsilon \cdot \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_i(1 - y_i)$$

更新 $W_{r,i}^{(2)}$

要更新 $W_{r,i}^{(2)}$, 我们就需要从 γ_j 继续写下去

$$y_j = \text{sigmoid}(\gamma_j)$$

$$\gamma_j = (b_1 \quad \cdots \quad b_i \quad \cdots \quad b_{20}) \begin{pmatrix} W_{1,j}^{(3)} \\ W_{2,j}^{(3)} \\ \cdot \\ \cdot \\ W_{20,j}^{(3)} \end{pmatrix} + Bias_j^{(3)}$$

$$b_i = \text{sigmoid}(\beta_i)$$

$$\beta_i = (a_1 \quad a_2 \quad \cdots \quad a_{20}) \begin{pmatrix} W_{1,i}^{(2)} \\ W_{2,i}^{(2)} \\ \cdot \\ \cdot \\ W_{20,i}^{(2)} \end{pmatrix} + Bias_i^{(2)}$$

这样以后, 梯度

$$\begin{aligned}
\nabla G &= \frac{\partial Loss}{\partial W_{r,i}^{(2)}} \\
&= \frac{\partial Loss}{\partial y_j} \frac{\partial y_j}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial b_i} \frac{\partial b_i}{\partial \beta_i} \frac{\partial \beta_i}{\partial W_{r,i}^{(2)}} \\
&= \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j (1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i (1 - b_i) \cdot a_r
\end{aligned}$$

因此，更新后 $W_{r,i}^{(2) '}$

$$W_{r,i}^{(2) '} = W_{r,i}^{(2)} - \epsilon \cdot \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j (1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i (1 - b_i) \cdot a_r$$

更新 $Bias_i^{(2)}$

参考上述步骤，

$$\begin{aligned}
\nabla G &= \frac{\partial Loss}{\partial Bias_i^{(2)}} \\
&= \frac{\partial Loss}{\partial y_j} \frac{\partial y_j}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial b_i} \frac{\partial b_i}{\partial \beta_i} \frac{\partial \beta_i}{\partial Bias_i^{(2)}} \\
&= \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j (1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i (1 - b_i) \cdot \mathbf{1}
\end{aligned}$$

因此更新完后的 $Bias_i^{(2) '}$ 应该为

$$Bias_i^{(2) '} = Bias_i^{(2)} - \epsilon \cdot \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j (1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i (1 - b_i) \cdot a_r$$

更新 $W_{l,r}^{(1)}$

继续列出式子，

$$y_j = \text{sigmoid}(\gamma_j)$$

$$\gamma_j = (b_1 \quad \cdots \quad b_i \quad \cdots \quad b_{20}) \begin{pmatrix} W_{1,j}^{(3)} \\ W_{2,j}^{(3)} \\ \cdot \\ \cdot \\ W_{20,j}^{(3)} \end{pmatrix} + \text{Bias}_j^{(3)}$$

$$b_i = \text{sigmoid}(\beta_i)$$

$$\beta_i = (a_1 \quad \cdots \quad a_r \quad \cdots \quad a_{20}) \begin{pmatrix} W_{1,i}^{(2)} \\ W_{2,i}^{(2)} \\ \cdot \\ \cdot \\ W_{20,i}^{(2)} \end{pmatrix} + \text{Bias}_i^{(2)}$$

$$a_r = \text{sigmoid}(\alpha_r)$$

$$\alpha_r = (x_1 \quad \cdots \quad x_{20}) \begin{pmatrix} W_{1,r}^{(1)} \\ W_{2,r}^{(1)} \\ \cdot \\ \cdot \\ W_{784,r}^{(1)} \end{pmatrix} + \text{Bias}_r^{(1)}$$

其梯度,

$$\begin{aligned} \nabla G &= \frac{\partial \text{Loss}}{\partial W_{l,r}^{(1)}} \\ &= \frac{\partial \text{Loss}}{\partial y_j} \frac{\partial y_j}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial b_i} \frac{\partial b_i}{\partial \beta_i} \frac{\partial \beta_i}{\partial a_r} \frac{\partial a_r}{\partial \alpha_r} \frac{\partial \alpha_r}{\partial W_{l,r}^{(1)}} \\ &= \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i) \cdot W_{r,i}^{(2)} \cdot a_r(1 - a_r) \cdot x_l \end{aligned}$$

因此,

$$W_{l,r}^{(1)'} = W_{l,r}^{(1)} - \epsilon \cdot \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i) \cdot W_{r,i}^{(2)} \cdot a_r(1 - a_r) \cdot x_l$$

更新 $Bias_r^{(1)}$

同理，列出式子

$$y_j = \text{sigmoid}(\gamma_j)$$

$$\gamma_j = (b_1 \quad \cdots \quad b_i \quad \cdots \quad b_{20}) \begin{pmatrix} W_{1,j}^{(3)} \\ W_{2,j}^{(3)} \\ \vdots \\ W_{20,j}^{(3)} \end{pmatrix} + Bias_j^{(3)}$$

$$b_i = \text{sigmoid}(\beta_i)$$

$$\beta_i = (a_1 \quad \cdots \quad a_r \quad \cdots \quad a_{20}) \begin{pmatrix} W_{1,i}^{(2)} \\ W_{2,i}^{(2)} \\ \vdots \\ W_{20,i}^{(2)} \end{pmatrix} + Bias_i^{(2)}$$

$$a_r = \text{sigmoid}(\alpha_r)$$

$$\alpha_r = (x_1 \quad \cdots \quad x_{20}) \begin{pmatrix} W_{1,r}^{(1)} \\ W_{2,r}^{(1)} \\ \vdots \\ W_{784,r}^{(1)} \end{pmatrix} + Bias_r^{(1)}$$

然后梯度

$$\begin{aligned} \nabla G &= \frac{\partial Loss}{\partial Bias_r^{(1)}} \\ &= \frac{\partial Loss}{\partial y_j} \frac{\partial y_j}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial b_i} \frac{\partial b_i}{\partial \beta_i} \frac{\partial \beta_i}{\partial a_r} \frac{\partial a_r}{\partial \alpha_r} \frac{\partial \alpha_r}{\partial Bias_r^{(1)}} \\ &= \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i) \cdot W_{r,i}^{(2)} \cdot a_r(1 - a_r) \cdot \mathbf{1} \end{aligned}$$

更新后 $Bias_r^{(1) '}$ 应当为

$$Bias_r^{(1) '} = Bias_r^{(1)} - \epsilon \cdot \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i) \cdot W_{r,i}^{(2)} \cdot a_r(1 - a_r)$$

因此，总的结果应当为：

权重/偏置	其梯度
$W_{i,j}^{(3)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_i (1 - y_i) \cdot b_i$
$Bias_j^{(3)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j (1 - y_j) \cdot \mathbf{1}$
$W_{r,i}^{(2)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j (1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i (1 - b_i) \cdot a_r$
$Bias_i^{(2)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j (1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i (1 - b_i) \cdot \mathbf{1}$
$W_{l,r}^{(1)}$	$\nabla G = \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j - y_{gt(j)}) \cdot y_j (1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i (1 - b_i) \cdot W_{r,i}^{(2)} \cdot a_r (1 - a_r) \cdot x_l$
$Bias_r^{(1)}$	$\nabla G = \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j - y_{gt(j)}) \cdot y_j (1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i (1 - b_i) \cdot W_{r,i}^{(2)} \cdot a_r (1 - a_r) \cdot \mathbf{1}$

对于系数这方面我们就不用考虑那么多，因为只是一个倍数的问题，后面的Adam优化器会自己调整大小的。

给权重添加正则化 L1 和 L2

为了防止过度拟合问题，在损失函数的基础上我们会添加L1和L2正则化项。其中，定义

$$\begin{aligned}\lambda_1 &= 0.001 \\ \lambda_2 &= 0.001\end{aligned}$$

那么损失函数就有

$$Loss = \frac{1}{2} \sum_{j=1}^{10} (y_j - y_{gt(j)})^2 + \lambda_1 \cdot \sum |W| + \lambda_2 \cdot \sum W^2$$

同时，损失函数纠纷为了这三个部分：均方差损失+L1损失+L2损失

$$\begin{aligned}Loss &= L_{MSE} + L_{L1} + L_{L2} \\ L_{MSE} &= \frac{1}{2} \sum_{j=1}^{10} (y_j - y_{gt(j)})^2 \\ L_{L1} &= \lambda_1 \cdot \sum |W| \\ L_{L2} &= \lambda_2 \cdot \sum W^2\end{aligned}$$

不过很显然，L1和L2正则化只对权重的梯度有影响，对偏置的梯度没有影响。

因此，我们用 $W_{i,j}^{(3)}$ 举例，来看看添加了正则化之后的梯度会变成什么样

$$\begin{aligned}\nabla G &= \frac{\partial Loss}{\partial W_{i,j}^{(3)}} \\ &= \frac{\partial L_{MSE}}{\partial W_{i,j}^{(3)}} + \frac{\partial L_{L1}}{\partial W_{i,j}^{(3)}} + \frac{\partial L_{L2}}{\partial W_{i,j}^{(3)}} \\ &= \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_i (1 - y_i) \cdot b_i + \lambda_1 \cdot \mathbf{sign}(W_{i,j}^{(3)}) + 2\lambda_2 \cdot W_{i,j}^{(3)}\end{aligned}$$

其中

$$\mathbf{sign}(x) = \begin{cases} -1 & , \text{if } x < 0 \\ 0 & , \text{if } x = 0 \\ 1 & , \text{if } x > 0 \end{cases}$$

因此，添加L1和L2正则化后的梯度应当为

权重/偏置	其梯度
$W_{i,j}^{(3)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_i(1 - y_i) \cdot b_i + \lambda_1 \cdot \text{sign}(W_{i,j}^{(3)}) + 2\lambda_2 \cdot W_{i,j}^{(3)}$
$Bias_j^{(3)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j)$
$W_{r,i}^{(2)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i) \cdot a_r + \lambda_1 \cdot \text{sign}(W_{r,i}^{(2)}) + 2\lambda_2 \cdot W_{r,i}^{(2)}$
$Bias_i^{(2)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i)$
$W_{l,r}^{(1)}$	$\nabla G = \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i) \cdot W_{r,i}^{(2)} \cdot a_r(1 - a_r) \cdot x_i + \lambda_1 \cdot \text{sign}(W_{l,r}^{(1)}) + 2\lambda_2 \cdot W_{l,r}^{(1)}$
$Bias_r^{(1)}$	$\nabla G = \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i) \cdot W_{r,i}^{(2)} \cdot a_r(1 - a_r)$

Adam优化器

为了解决震荡问题，和学习率过大而错过最小值的问题，Adam优化器给出了调整方案。

以往的梯度一般是用固定的学习率，例如 $\epsilon = 0.001$ ，但是梯度缩小后学习率也应当相应缩小。同时要解决震荡问题。

$$W_{i,j}^{(3)'} = W_{i,j}^{(3)} - \epsilon \cdot \nabla G$$

因此，接下来我来介绍Adam优化器的步骤：

1. 算出梯度 ∇G
2. $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla G_t$
3. $v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \nabla G_t^2$
4. $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$
5. $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$
6. 更新 $W_{i,j}^{(3)'} = W_{i,j}^{(3)} - \frac{\alpha \cdot \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$

说明：在这里，有四个常数

常数项	$\beta_1 = 0.9$
常数项	$\beta_2 = 0.999$
为了防止 $\sqrt{\hat{v}_t}$ 为0的情况而添加一个非常小的值	$\epsilon = 10^{-8}$
学习率	$\alpha = 0.001$

此外，

1. 下标 t 代表的是迭代第 t 次。
2. m_0, v_0 初始值都为 0。

随机梯度下降

在先前，我们知道了梯度

权重/偏置	其梯度
$W_{i,j}^{(3)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_i(1 - y_i) \cdot b_i + \lambda_1 \cdot \text{sign}(W_{i,j}^{(3)}) + 2\lambda_2 \cdot W_{i,j}^{(3)}$
$Bias_j^{(3)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j)$
$W_{r,i}^{(2)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i) \cdot a_r + \lambda_1 \cdot \text{sign}(W_{r,i}^{(2)}) + 2\lambda_2 \cdot W_{r,i}^{(2)}$
$Bias_i^{(2)}$	$\nabla G = \sum_{j=1}^{10} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i)$

权重/偏置	其梯度
$W_{l,r}^{(1)}$	$\nabla G = \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i) \cdot W_{r,i}^{(2)} \cdot a_r(1 - a_r) \cdot x_l + \lambda_1 \cdot \text{sign}(W_{l,r}^{(1)}) + 2\lambda_2 \cdot W_{l,r}^{(1)}$
$Bias_r^{(1)}$	$\nabla G = \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j - y_{gt(j)}) \cdot y_j(1 - y_j) \cdot W_{i,j}^{(3)} \cdot b_i(1 - b_i) \cdot W_{r,i}^{(2)} \cdot a_r(1 - a_r)$

但是我们不得不思考一个问题。假设我们要训练60000个图像。如果我们还是使用原始的方法，要进行梯度下降，那么就需要遍历60000个图像，然后把这些权重的值加起来做平均，才得到一个总梯度。这时候才下降了一次。如下图

							...	Average over all training data
w_0	-0.08	+0.02	-0.02	+0.11	-0.05	-0.14	...	→ -0.08
w_1	-0.11	+0.11	+0.07	+0.02	+0.09	+0.05	...	→ +0.12
w_2	-0.07	-0.04	-0.01	+0.02	+0.13	-0.15	...	→ -0.06
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$w_{13,001}$	+0.13	+0.08	-0.06	-0.09	-0.02	+0.04	...	→ +0.04

图像来源: [3B1B - 深度学习之反向传播算法 上/下 Part 3 ver 0.9 beta]

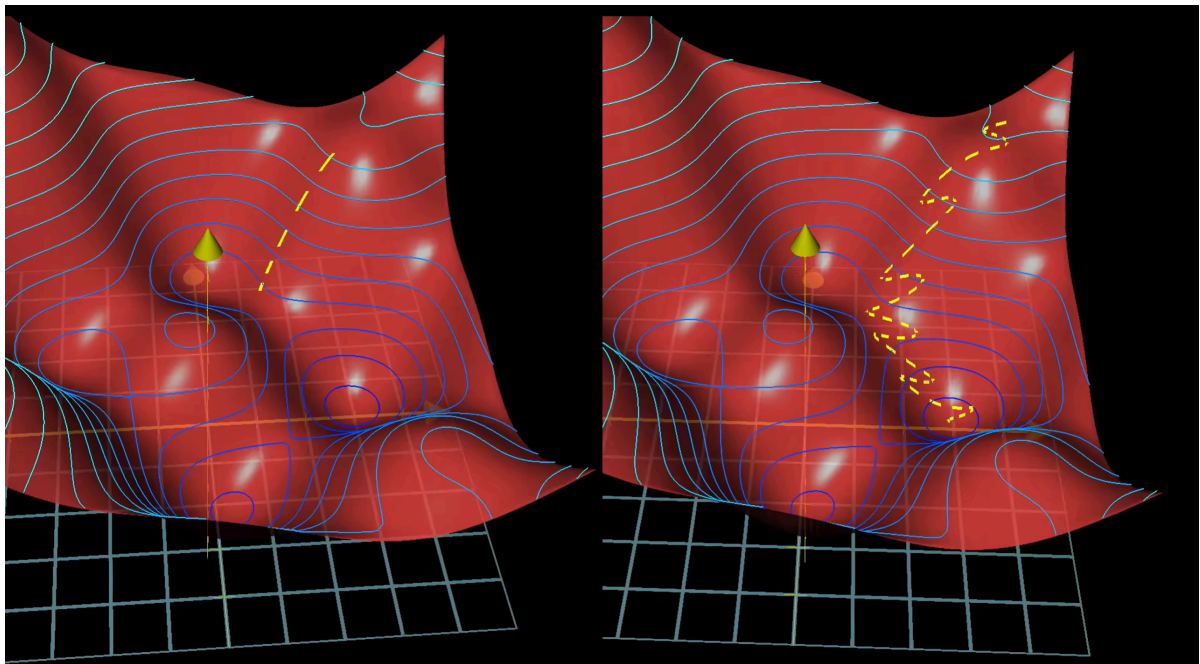
[<https://www.bilibili.com/video/BV16x411V7Qg>]

但是这样做的弊端就是，下降得太慢了！我们遍历了60000个图像，才下降了一次！虽然这样下降很精准，但是很慢，而且由于60000个图像需要全部加载到内存中，这样对内存非常不友好。

因此，我们可以引入批次batch的概念。例如，在60000个图像中随机抽取64个图像作为一个batch，这样我们只需要求出这64个图像的权重变化均值后，就可以对网络进行一次下降了！而且内存也会随之减少。

当然，这时候一轮Epoch就相当于要进行完 $60000/64 \approx 938$ 次大小为64的batch的下降。同时，一轮Epoch就相当于比原来快了938倍。

左图是标准的梯度下降，很慢，但是很精确；右图是随机梯度下降，也就是引入批次的方法，虽然下降不怎么精确，但是很快，也可以到达最低点。



图像来源: [3B1B - 深度学习之反向传播算法 上/下 Part 3 ver 0.9 beta]

[https://www.bilibili.com/video/BV16x411V7Qg]

但是实际上在这个程序中并没有设置batch的必要性，因为实测中两者的时间和内存以及结果上看别无二致。

如果单个数据集不是 28×28 而是更大的话，就可以考虑到引入批次的概念。

引入批次后，假设 $\text{batch} = 64$ ，那么梯度就应当更改为，这个批次下所有图像梯度平均。

权重/偏置	其梯度
$W_{i,j}^{(3)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_i^{(n)} (1 - y_i^{(n)}) \cdot b_i^{(n)} + \lambda_1 \cdot \text{sign}(W_{i,j}^{(3)}) + 2\lambda_2 \cdot W_{i,j}^{(3)}$
$Bias_j^{(3)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)})$
$W_{r,i}^{(2)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)}) \cdot a_r^{(n)} + \lambda_1 \cdot \text{sign}(W_{r,i}^{(2)}) + 2\lambda_2 \cdot W_{r,i}^{(2)}$
$Bias_i^{(2)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)})$
$W_{l,r}^{(1)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)}) \cdot W_{r,i}^{(2)} \cdot a_r^{(n)} (1 - a_r^{(n)}) \cdot x_l^{(n)} + \lambda_1 \cdot \text{sign}(W_{l,r}^{(1)}) + 2\lambda_2 \cdot W_{l,r}^{(1)}$
$Bias_r^{(1)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)}) \cdot W_{r,i}^{(2)} \cdot a_r^{(n)} (1 - a_r^{(n)})$

在这里， x, a, b, y 的上标 (n) 代表批次中的第 n 个图像。

例如， $x_{740}^{(52)}$ 代表第52个图像的输入层X的第740个值。

代码实现

首先注意到，当更新最后一层权重，也就是 $W_{l,r}^{(1)}$ 的时候，会涉及到5层循环，这会大大降低代码运行效率。然而，权重和偏置的梯度是又很多地方重复的。

例如，在考虑这个问题的时候，我们先移除正则化对梯度的影响。会发现所有的偏置对权重而言都只是把最后一项换成了1。因此，我们可以先求偏置后求权重。

在开始之前，我们可以考虑把1~64的X,A,B,Y这四层预先存储到一个 `double[][][]` 中。

输入一个 `ArrayList<double[][]> batch` 代表这个batch中有 `batch.size()` 个 29×28 像素的图片

其中，`batch.get(n)[28][0]` 是第 n 个图像的标签。

`xList[52][740]` 就是 $x_{740}^{(52)}$ 的值。


```

private static void trainBatch(ArrayList<double[][]> batch) {
    double[][] XList = new double[batch.size()][784];
    double[][] AList = new double[batch.size()][20];
    double[][] BList = new double[batch.size()][20];
    double[][] YList = new double[batch.size()][10];

    for (int numberOfBatch = 0; numberOfBatch < batch.size(); num
double[] X = new double[784];
double[] A = new double[20];
double[] B = new double[20];
double[] Y = new double[10];
double[][] currentImg = batch.get(numberOfBatch);
// 把二维数组图像转存为一维数组
int counter = 0;
    for (int numberOfBatch = 0; numberOfBatch < batch.size();
numberOfBatch++) {
        for (int y = 0; y < 28; y++) {
            X[counter++] = currentImg[x][y];
        }
    }
    // 计算隐藏层A
    for (int i = 0; i < 20; i++) {
        for (int left = 0; left < 784; left++) {
            A[i] += X[left] * Processing.weightXA[left][i];
        }
        // 添加偏置后, 压入sigmoid
        A[i] += Processing.biasA[i];
        A[i] = sigmoid(A[i]);
    }
    // 计算隐藏层B
    for (int i = 0; i < 20; i++) {
        for (int left = 0; left < 20; left++) {
            B[i] += A[left] * Processing.weightAB[left][i];
        }
        B[i] += Processing.biasB[i];
        B[i] = sigmoid(B[i]);
    }
    // 计算Y
    for (int i = 0; i < 10; i++) {
        for (int left = 0; left < 20; left++) {
            Y[i] += B[left] * Processing.weightBY[left][i];
        }
        Y[i] += Processing.biasY[i];
        Y[i] = sigmoid(Y[i]);
    }
    XList[numberOfBatch] = arrayCopy(X);
    AList[numberOfBatch] = arrayCopy(A);
    BList[numberOfBatch] = arrayCopy(B);
    YList[numberOfBatch] = arrayCopy(Y);
}

// 这里还有后续代码.....
}

```

计算梯度

首先我们先算出梯度。在 `trainBatch` 方法的开头我们同时也定义

```
double[][] gradient_weightXA = new double[784][20];
double[][] gradient_weightAB = new double[20][20];
double[][] gradient_weightBY = new double[20][10];
double[] gradient_biasA = new double[20];
double[] gradient_biasB = new double[20];
double[] gradient_biasY = new double[10];
```

这时候观察表格：

权重/偏置	其梯度
$W_{i,j}^{(3)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot b_i^{(n)} + \lambda_1 \cdot \text{sign}(W_{i,j}^{(3)}) + 2\lambda_2 \cdot W_{i,j}^{(3)}$
$Bias_j^{(3)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)})$
$W_{r,i}^{(2)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)}) \cdot a_r^{(n)} + \lambda_1 \cdot \text{sign}(W_{r,i}^{(2)}) + 2\lambda_2 \cdot W_{r,i}^{(2)}$
$Bias_i^{(2)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)})$
$W_{l,r}^{(1)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)}) \cdot W_{r,i}^{(2)} \cdot a_r^{(n)} (1 - a_r^{(n)}) \cdot x_l^{(n)} + \lambda_1 \cdot \text{sign}(W_{l,r}^{(1)}) + 2\lambda_2 \cdot W_{l,r}^{(1)}$
$Bias_r^{(1)}$	$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)}) \cdot W_{r,i}^{(2)} \cdot a_r^{(n)} (1 - a_r^{(n)})$

发现可以把 $\sum_{n=1}^{64}$ 提出来，然后写入一个循环框架

```
for (int n = 0; n < batch.size(); n++) {
    // 下降 biasY
    // 下降 weightBY
    // 下降 biasB
    // 下降 weightAB
    // 下降 biasA
    // 下降 weightXA
}
```

首先我们把第 `n` 个图像的正确标签提取出来

```
double label = batch.get(n)[28][0];
```

第一步，计算 `gradient_biasY`

$$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)})$$

根据公式，写出：

```
// 下降 biasY
for (int j = 0; j < 10; j++) {
    double ygt; // 实际值
    if (label == j) ygt = 1.0;
    else ygt = 0.0;
    double value = (YList[n][j] - ygt) * YList[n][j] * (1.0 - YList[n][j]);
    gradient_biasY[j] += value;
    storage1.add(value);
}
```

在这里我们每次计算都存入了 `ArrayList<Double> storage1 = new ArrayList<>()`

这样可以方便后续的计算!

第二步, 计算 `gradient_weightBY`

$$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_i^{(n)} (1 - y_i^{(n)}) \cdot b_i^{(n)}$$

```
// weightBY
for (int i = 0; i < 20; i++) {
    for (int j = 0; j < 10; j++) {
        double value = storage1.get(j) * BList[n][i]; // 因为重复部分可以直接调用!
        gradient_weightBY[i][j] += value;
    }
}
```

第三步, 计算 `gradient_biasB`

$$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)})$$

可见, 对于 **weightBY**而言, 仅仅比**biasB**多了2项。因此我们可以在 **weightBY**的代码中添加存到 `ArrayList<Double> storage2 = new ArrayList<>()` 的方法。

```
// weightBY(更新后的代码)
for (int i = 0; i < 20; i++) {
    double temp = 0; // 添加temp
    for (int j = 0; j < 10; j++) {
        double value = storage1.get(j) * BList[n][i];
        gradient_weightBY[i][j] += value;

        temp += storage1.get(j) * Processing.weightBY[i][j] * BList[n][i] * (1.0 - BList[n][i]); // 累加
    }
    storage2.add(temp);
}
```

因此, **biasB** 直接写成

```
// biasB
for (int i = 0; i < 20; i++) {
    gradient_biasB[i] += storage2.get(i);
}
```

第四步, 计算 `gradient_weightAB`

$$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)}) \cdot a_r^{(n)}$$

显然 **weightAB**仅仅比 **biasB** 多了一项 $a_r^{(n)}$

因此我们直接调用 storage2 即可！

```
// weightAB
for (int r = 0; r < 20; r++) {
    for (int i = 0; i < 20; i++) {
        gradient_weightAB[r][i] += storage2.get(i) * AList[n][r];
    }
}
```

第五步，计算 gradient_biasA

$$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)}) \cdot W_{r,i}^{(2)} \cdot a_r^{(n)} (1 - a_r^{(n)})$$

对于 **biasA**，多出两项，因此我们可以在 weightAB 的代码中添加一些修改，让 weightAB 的每次累加结果存到 storage3 中

```
// weightAB
for (int r = 0; r < 20; r++) {
    double temp = 0;
    for (int i = 0; i < 20; i++) {
        gradient_weightAB[r][i] += storage2.get(i) * AList[n][r];
        temp += storage2.get(i) * AList[n][r] * weightAB[r][i] * (1.0 - AList[n][r]);
    }
    storage3.add(temp);
}
```

因此对于 **biasA** 我们就直接这样写

```
// biasA
for (int r = 0; r < 20; r++) {
    gradient_biasA[r] += storage3.get(r);
}
```

第六步，计算 gradient_weightXA

$$\nabla G = \sum_{n=1}^{64} \sum_{j=1}^{10} \sum_{i=1}^{20} (y_j^{(n)} - y_{gt(j)}^{(n)}) \cdot y_j^{(n)} (1 - y_j^{(n)}) \cdot W_{i,j}^{(3)} \cdot b_i^{(n)} (1 - b_i^{(n)}) \cdot W_{r,i}^{(2)} \cdot a_r^{(n)} (1 - a_r^{(n)}) \cdot x_l^{(n)}$$

按照这个逻辑，我们就可以写出

```
// weightXA
for (int l = 0; l < 784; l++) {
    for (int r = 0; r < 20; r++) {
        gradient_weightXA[l][r] += storage3.get(r) * xList[n][l];
    }
}
```

添加正则化项

接下来添加正则化项！只需要针对三个权重添加即可

```
// gradient_weightBY
for (int i = 0; i < 20; i++) {
    for (int j = 0; j < 10; j++) {
        gradient_weightBY[i][j] += 2 * LAMBDA_L2 * Processing.weightBY[i][j] +
        LAMBDA_L1 * sign(Processing.weightBY[i][j]);
    }
}

// gradient_weightAB
for (int right = 0; right < 20; right++) {
    for (int i = 0; i < 20; i++) {
        gradient_weightAB[right][i] += 2 * LAMBDA_L2 * Processing.weightAB[right]
[i] + LAMBDA_L1 * sign(Processing.weightAB[right][i]);
    }
}

// gradient_weightXA
for (int left = 0; left < 784; left++) {
    for (int right = 0; right < 20; right++) {
        gradient_weightXA[left][right] += 2 * LAMBDA_L2 *
Processing.weightXA[left][right] + LAMBDA_L1 * sign(Processing.weightXA[left]
[right]);
    }
}
```

Adam优化器进行梯度下降

接下来，我们使用Adam优化器进行梯度下降

Adam优化器的步骤：

1. 算出梯度 ∇G
2. $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla G_t$
3. $v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \nabla G_t^2$
4. $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$
5. $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$
6. 更新 $x' = x - \frac{\alpha \cdot \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$

首先，在全局中定义变量如下

```
static double BETA1 = 0.9;
static double BETA2 = 0.999;
static double ALPHA = 0.001; // 学习率
static double EPSILON = 1.0E-08; // 为了防止分母为0
static double LAMBDA_L1 = 0.001; // 正则化L1 参数
static double LAMBDA_L2 = 0.001; // 正则化L2 参数
// weightXA 的 Adam优化器数据
static double[][] weightXA_Mt = new double[784][20];
static double[][] weightXA_vt = new double[784][20];
```

```

static int weightXA_Iteration = 0;
// weightAB 的 Adam优化器数据
static double[][] weightAB_Mt = new double[20][20];
static double[][] weightAB_Vt = new double[20][20];
static int weightAB_Iteration = 0;
// weightBY 的 Adam优化器数据
static double[][] weightBY_Mt = new double[20][10];
static double[][] weightBY_Vt = new double[20][10];
static int weightBY_Iteration = 0;
// biasA 的 Adam优化器数据
static double[] biasA_Mt = new double[20];
static double[] biasA_Vt = new double[20];
static int biasA_Iteration = 0;
// biasB 的 Adam优化器数据
static double[] biasB_Mt = new double[20];
static double[] biasB_Vt = new double[20];
static int biasB_Iteration = 0;
// biasY 的 Adam优化器数据
static double[] biasY_Mt = new double[20];
static double[] biasY_Vt = new double[20];
static int biasY_Iteration = 0;

```

在方法处，我们需要将默认迭代数自增1。

```

++weightXA_Iteration;
++weightAB_Iteration;
++weightBY_Iteration;
++biasA_Iteration;
++biasB_Iteration;
++biasY_Iteration;

```

因此，代码应当是

1. 更新 biasY

```

// biasY
for (int j = 0; j < 10; j++) {
    biasY_Mt[j] = BETA1 * biasY_Mt[j] + (1.0 - BETA1) * gradient_biasY[j];
    biasY_Vt[j] = BETA2 * biasY_Vt[j] + (1.0 - BETA2) * gradient_biasY[j] *
gradient_biasY[j];
    double MtHat = biasY_Mt[j] / (1.0 - Math.pow(BETA1, biasY_Iteration));
    double VtHat = biasY_Vt[j] / (1.0 - Math.pow(BETA2, biasY_Iteration));
    biasY[j] = biasY[j] - (ALPHA * MtHat) / (Math.sqrt(VtHat) + EPSILON);
}

```

2. 更新 weight_BY

```
// weight_BY
for (int i = 0; i < 20; i++) {
    for (int j = 0; j < 10; j++) {
        weightBY_Mt[i][j] = BETA1 * weightBY_Mt[i][j] + (1.0 - BETA1) *
gradient_weightBY[i][j];
        weightBY_Vt[i][j] = BETA2 * weightBY_Vt[i][j] + (1.0 - BETA2) *
gradient_weightBY[i][j] * gradient_weightBY[i][j];
        double MtHat = weightBY_Mt[i][j] / (1.0 - Math.pow(BETA1,
weightBY_Iteration));
        double VtHat = weightBY_Vt[i][j] / (1.0 - Math.pow(BETA2,
weightBY_Iteration));
        weightBY[i][j] = weightBY[i][j] - (ALPHA * MtHat) / (Math.sqrt(VtHat) +
EPSILON);
    }
}
```

3. 更新 biasB

```
// biasB
for (int i = 0; i < 20; i++) {
    biasB_Mt[i] = BETA1 * biasB_Mt[i] + (1.0 - BETA1) * gradient_biasB[i];
    biasB_Vt[i] = BETA2 * biasB_Vt[i] + (1.0 - BETA2) * gradient_biasB[i] *
gradient_biasB[i];
    double MtHat = biasB_Mt[i] / (1.0 - Math.pow(BETA1, biasB_Iteration));
    double VtHat = biasB_Vt[i] / (1.0 - Math.pow(BETA2, biasB_Iteration));
    biasB[i] = biasB[i] - (ALPHA * MtHat) / (Math.sqrt(VtHat) + EPSILON);
}
```

4. 更新 weightAB

```
// weightAB
for (int right = 0; right < 20; right++) {
    for (int i = 0; i < 20; i++) {
        weightAB_Mt[right][i] = BETA1 * weightAB_Mt[right][i] + (1.0 - BETA1) *
gradient_weightAB[right][i];
        weightAB_Vt[right][i] = BETA2 * weightAB_Vt[right][i] + (1.0 - BETA2) *
gradient_weightAB[right][i] * gradient_weightAB[right][i];
        double MtHat = weightAB_Mt[right][i] / (1.0 - Math.pow(BETA1,
weightAB_Iteration));
        double VtHat = weightAB_Vt[right][i] / (1.0 - Math.pow(BETA2,
weightAB_Iteration));
        weightAB[right][i] = weightAB[right][i] - (ALPHA * MtHat) /
(Math.sqrt(VtHat) + EPSILON);
    }
}
```

5. 更新 biasA

```
// biasA
for (int right = 0; right < 20; right++) {
    biasA_Mt[right] = BETA1 * biasA_Mt[right] + (1.0 - BETA1) *
gradient_biasA[right];
    biasA_Vt[right] = BETA2 * biasA_Vt[right] + (1.0 - BETA2) *
gradient_biasA[right] * gradient_biasA[right];
    double MtHat = biasA_Mt[right] / (1.0 - Math.pow(BETA1, biasA_Iteration));
    double VtHat = biasA_Vt[right] / (1.0 - Math.pow(BETA2, biasA_Iteration));
    biasA[right] = biasA[right] - (ALPHA * MtHat) / (Math.sqrt(VtHat) + EPSILON);
}
```

6. 更新 weightXA

```
// weightXA
for (int left = 0; left < 784; left++) {
    for (int right = 0; right < 20; right++) {
        weightXA_Mt[left][right] = BETA1 * weightXA_Mt[left][right] + (1.0 -
BETA1) * gradient_weightXA[left][right];
        weightXA_Vt[left][right] = BETA2 * weightXA_Vt[left][right] + (1.0 -
BETA2) * gradient_weightXA[left][right] * gradient_weightXA[left][right];
        double MtHat = weightXA_Mt[left][right] / (1.0 - Math.pow(BETA1,
weightXA_Iteration));
        double VtHat = weightXA_Vt[left][right] / (1.0 - Math.pow(BETA2,
weightXA_Iteration));
        weightXA[left][right] = weightXA[left][right] - (ALPHA * MtHat) /
(Math.sqrt(VtHat) + EPSILON);
    }
}
```

到这里，反向传播就完成了。

因此，假设我们选了 `GUI.BATCH_SET_SIZE` 个数据作为总样本，`DEFINE_BATCH_SIZE` 作为一个batch的大小，那么接下来的两个循环结束就代表着一轮Epoch完成。

```
for (int i = 0; i < GUI.BATCH_SET_SIZE / DEFINE_BATCH_SIZE; i++) {
    for (int j = 0; j < DEFINE_BATCH_SIZE; j++) {
        // 投入
        int randomIndex = rand.nextInt(GUI.BATCH_SET_SIZE);
        batch.add(allImg.get(randomIndex));
    }
    trainBatch(batch);
    batch.clear();
}
```